

วิศวกรรมซอฟต์แวร์ (Software Engineering)

□ กลวิธีการทดสอบซอฟต์แวร์

กลวิธีการทดสอบซอฟต์แวร์

- **Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.**
- หลักการและเป้าหมายของการทดสอบ คือ การหาความผิดพลาดให้พบ ตัวทดสอบที่ดีควรมีความเป็นได้สูงที่จะหาข้อผิดพลาดพบ

ลักษณะของซอฟต์แวร์ที่ทดสอบได้ (Testability)

- ❑ ความสามารถในการทำงาน (Operability) – ยิ่งทำงานได้ดีเท่าไร ก็ยิ่งง่ายต่อการทดสอบเท่านั้น
- ❑ ความสามารถในการสังเกตเห็น (Observability) – สิ่งที่เราเห็นคือสิ่งที่ทดสอบได้
- ❑ ความสามารถในการควบคุม (Controllability) – ยิ่งเราสามารถควบคุมซอฟต์แวร์ได้ดีเท่าไร เราจะสามารถทดสอบโดยอัตโนมัติและเหมาะสมได้มากเท่านั้น
- ❑ ความสามารถในการแบ่งเป็นส่วนย่อย (Decomposability) – การควบคุมขอบเขตการทดสอบ ทำให้สามารถแยกแยะปัญหาและทำการทดสอบได้อย่างรวดเร็ว

ลักษณะของซอฟต์แวร์ที่ทดสอบได้ (Testability)

- ❑ ความเรียบง่าย (Simplicity) – ยิงมีสิ่งที่จะต้องทดสอบน้อยเท่าไร ยิงทดสอบได้เร็วเท่านั้น
- ❑ ความคงตัว (Stability) – การเปลี่ยนแปลงยิงน้อย ความซับซ้อนในการทดสอบก็ยิงน้อย
- ❑ ความสามารถในการเข้าใจ (Understandability) – ยิงเรารู้มากเท่าไร เราก้ยิงทดสอบอย่างฉลาดได้มากเท่านั้น

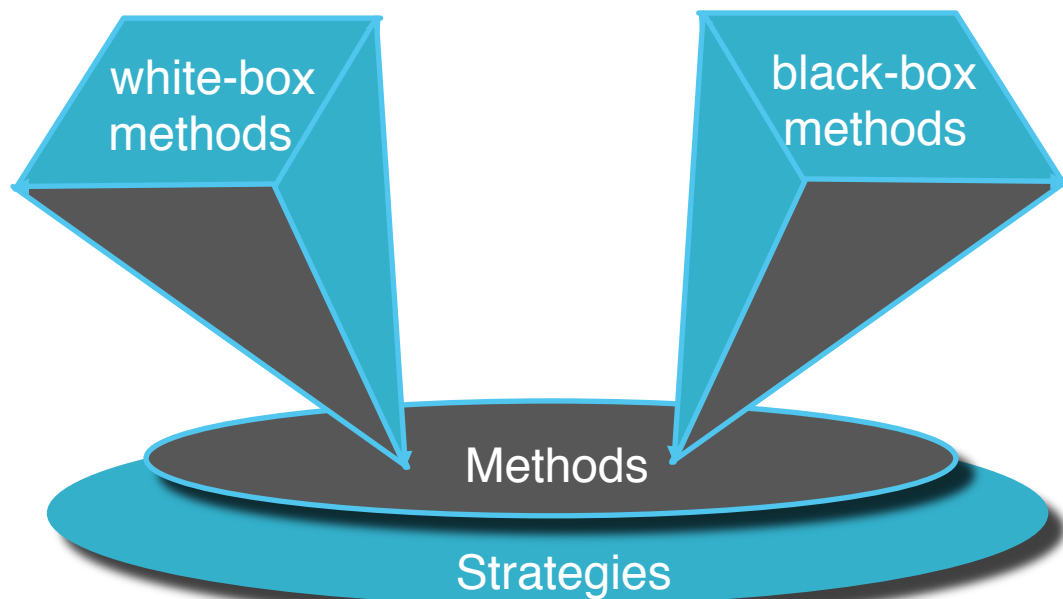
ลักษณะการทดสอบ (Test Characteristics)

ลักษณะของตัวทดสอบที่ดี

- ❑ A good test has a high probability of finding an error -- มีความน่าจะเป็นสูงในการหาความผิดพลาดพบ
- ❑ A good test is not redundant. – ไม่ซ้ำซ้อน
- ❑ A good test should be “best of breed” -- ควรเป็นตัวที่คิดมาดีที่สุด
- ❑ A good test should be neither too simple nor too complex – ไม่ธรรมดาเกินไปหรือซับซ้อนเกินไป

การทดสอบซอฟต์แวร์

- ❑ การทดสอบแบบกล่องขาว (white box) -- เป็นการทดสอบแบบละเอียดถึงกระบวนการทำงานภายใน
- ❑ การทดสอบแบบกล่องดำ (Black box) – ทำกับระดับอินเตอร์เฟซของซอฟต์แวร์



การทดสอบแบบกล่องขาว (white box)

- อาจเรียกว่า “การทดสอบแบบกล่องแก้ว (Glass box testing)” -- เป็นวิธีที่ใช้เส้นทางควบคุมการทำงานและโครงสร้างควบคุมที่ได้จากการออกแบบ มาช่วยออกแบบกรณีทดสอบ เพื่อทดสอบสิ่งต่อไปนี้
 - ▣ ทดสอบว่าทุกเส้นทางในกระบวนการจะต้องสามารถทำงานได้อย่างถูกต้อง
 - ▣ ทดสอบการตัดสินใจทางตรรกะทุกการตัดสินใจ ทั้งค่าจริงและเท็จ
 - ▣ ทดสอบการทำงานภายในลูปทุกลูปตามจำนวนของการวนรอบ
 - ▣ ทดสอบโครงสร้างข้อมูลภายในให้ถูกต้องก่อนส่งไปประมวลผลยังหน่วยอื่น

การทดสอบแบบกล่องขาว (white box)

- ❑ การทดสอบเส้นทางมูลฐาน (Basic path testing) เป็นเทคนิคการทดสอบแบบกล่องขาว ช่วยนักออกแบบกรณีทดสอบสร้างตัววัดความซับซ้อนของโมดูล และใช้ค่าที่วัดได้เป็นเกณฑ์ในการกำหนดชุดมูลฐานของเส้นทางทำงาน
- ❑ เพื่อทดสอบความสลับซับซ้อนทางตรรกะวิทยาโดยจะทดสอบทุกๆ “Execution path” การออกแบบกรณีทดสอบจะทำการทดสอบ “Basic set” ที่ใช้ “Execute logical path” ต่างๆ คือ

1) Sequence logical path

2) If logical path

3) Loop logical path

รวมถึงการทดสอบ Logical paths ที่ซับซ้อนขึ้น คือการออกแบบกรณีทดสอบที่รวมทุก Logical paths ข้างต้นเข้าด้วยกันเพื่อทำการทดสอบ

Basis Path Testing

- การทดสอบแบบ Basis Path จะต้องรู้จำนวนเส้นทางความซับซ้อนของปัญหา (cyclomatic complexity หรือแทนด้วยกราฟ $V(G)$) โดย $V(G)$ สามารถหาได้จาก 3 วิธี ดังนี้

วิธีที่ 1.

$V(G)$ = จำนวนของพื้นที่แบบปิด (enclosed area) ของผังงานโปรแกรม +1

วิธีที่ 2

$V(G)$ = จำนวนของสัญลักษณ์ตัดสินใจในผังงานโปรแกรม (simple decisions) +1

วิธีที่ 3

$V(G)$ = Edge (จำนวนเส้นเชื่อมระหว่าง Node) – Node (จำนวน Node ภายในกราฟ) +2

Basis Path Testing

วิธีที่ 1.

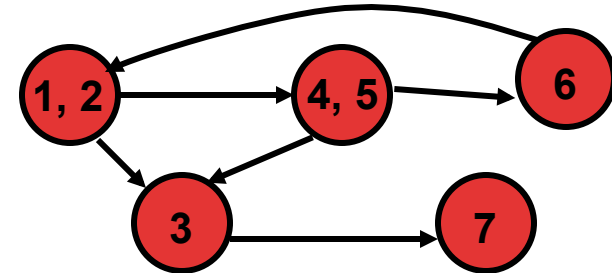
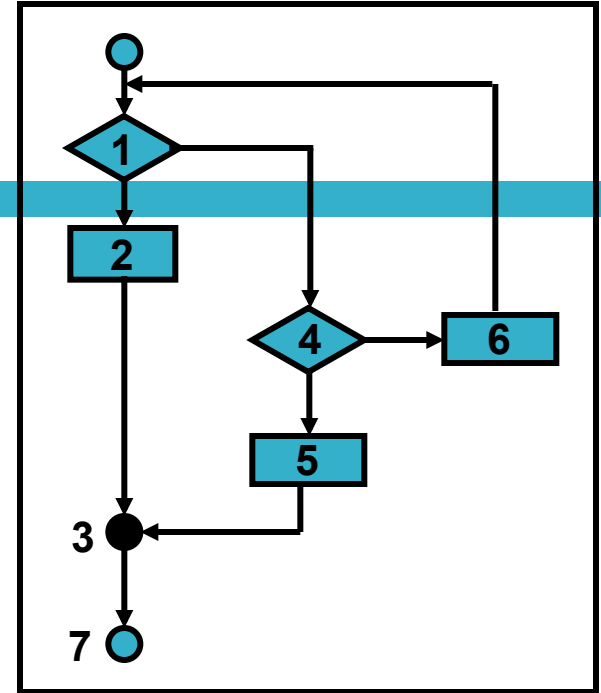
$V(G)$ = จำนวนของพื้นที่แบบปิด (enclosed area) ของ
ผังงานโปรแกรม + 1 ; $V(G) = 2 + 1 = 3$

วิธีที่ 2

$V(G)$ = จำนวนของสัญลักษณ์ตัดสินใจในผังงาน
โปรแกรม (simple decisions) + 1 ; $V(G) = 2 + 1 = 3$

วิธีที่ 3

$V(G) = \text{Edge (จำนวนเส้นเชื่อมระหว่าง Node)} -$
 $\text{Node (จำนวน Node ภายในกราฟ)} + 2$; $V(G) = 6 - 5$
 $+ 2 = 3$



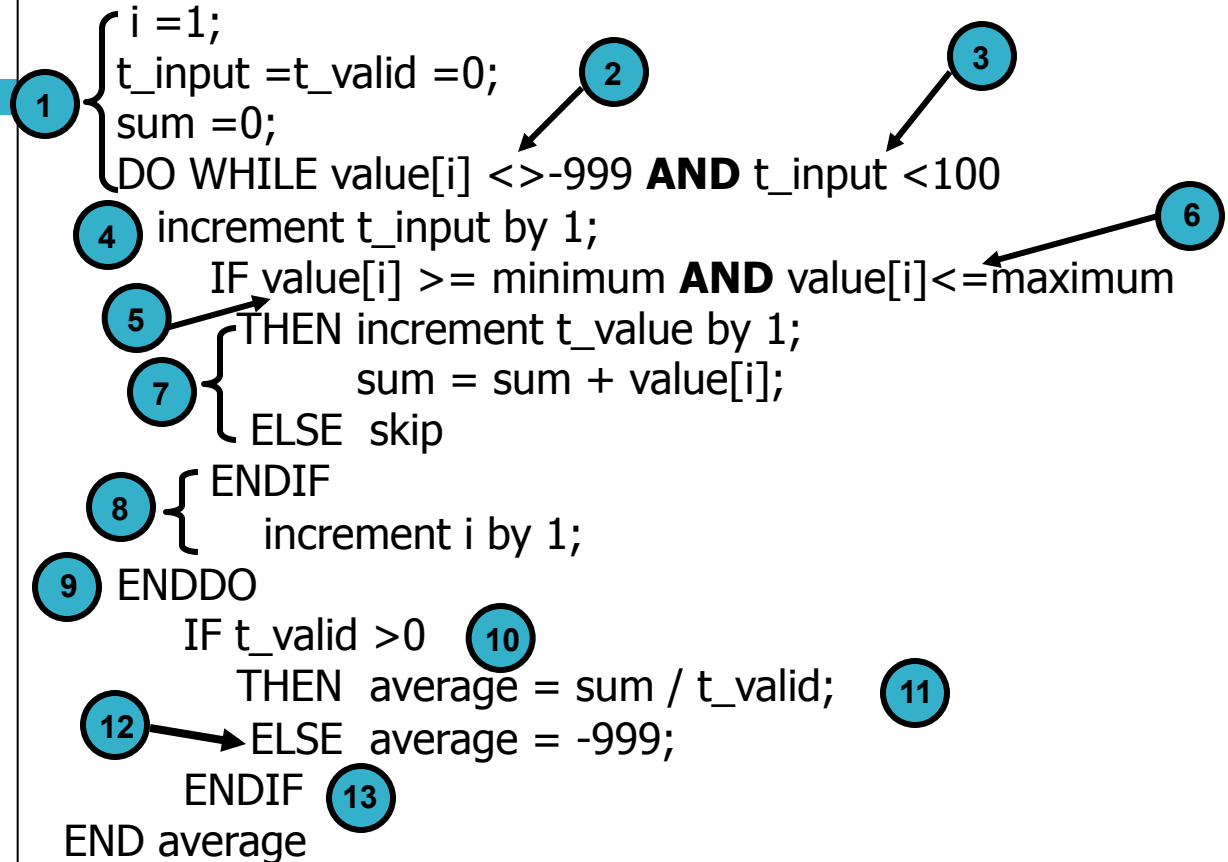
Basis Path Testing

- ❑ ค่าของเส้นทางความซับซ้อนของปัญหา $V(G)$ จะทำให้ได้เส้นทางเพื่อนำมา กำหนดในการทดสอบ
- ❑ การหา $V(G)$ สามารถเลือกใช้วิธีใดวิธีหนึ่งเพื่อให้ได้จำนวนเส้นทางที่จะทดสอบ

ตัวอย่าง

□ หาโหมดจาก
โปรแกรม

PROCEDURE average:



ตัวอย่าง

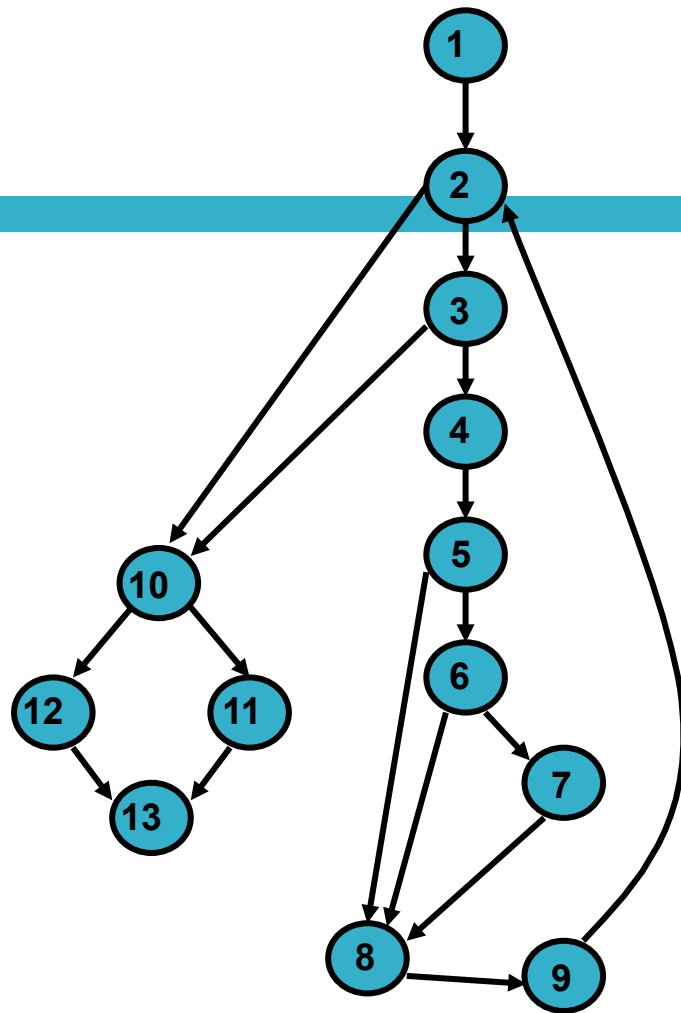
□ นำโหนดที่ได้มาวาดเป็นกราฟ

หาค่า $V(G)$

□ วิธีที่ 1 = $5+1=6$

□ วิธีที่ 2 = $5+1=6$

□ วิธีที่ 3 = $17-13+2=6$



พิจารณาเส้นทางที่แยกเป็นอิสระ (independent path)

- เส้นทางอิสระ (independent path) เป็นเส้นทางใดๆ ผ่านโปรแกรมที่มีประโยคคำสั่งหรือประโยคทดสอบเงื่อนไขอย่างน้อย 1 ประโยค
- จำนวนเส้นทางที่แยกเป็นอิสระ ได้จากค่า $V(G)$

independent path

จำนวนเส้นทางที่แยกเป็นอิสระ = 6 เส้นทาง
แต่ละเส้นทางมีรายละเอียดดังนี้

path 1: 1-2-10-11-13

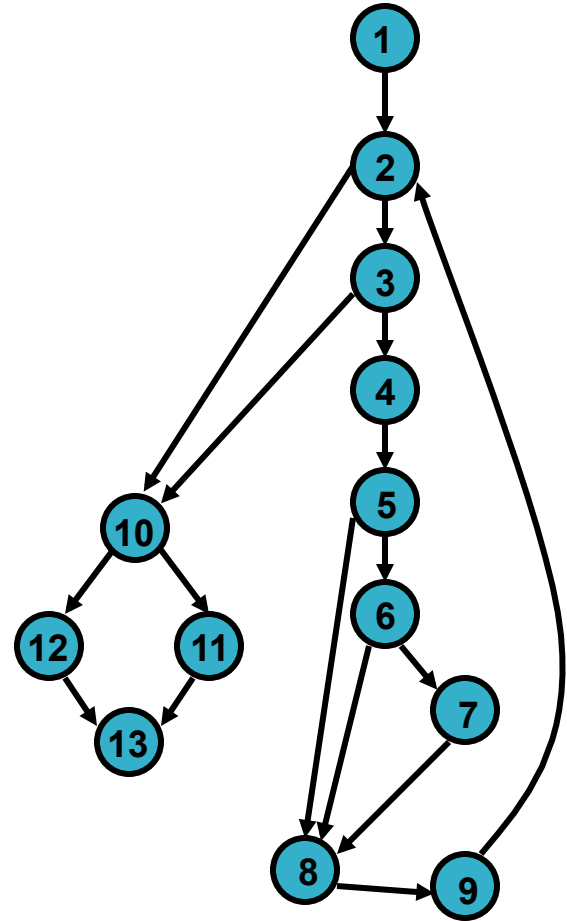
path 2: 1-2-10-12-13

path 3: 1-2-3-10-11-13

path 4: 1-2-3-4-5-8-9-2-...

path 5: 1-2-3-4-5-6-8-9-2-...

path 6: 1-2-3-4-5-6-7-8-9-2-...



การสร้างกรณีทดสอบ (test case)

- ❑ สร้างกรณีทดสอบ (test case) ที่จะนำมาทดสอบในแต่ละ path เลือกข้อมูลให้มีค่าเหมาะสมในการทดสอบแต่ละเส้นทาง
- ❑ เมื่อกำหนดกรณีทดสอบให้กับทุก path แล้วให้ทำการทดสอบแต่ละ path และเปรียบเทียบผลลัพธ์ที่ได้ กับ ผลลัพธ์ที่คาดหวัง (Expected results)

Loop Testing

□ เป็นเทคนิคในการทดสอบแบบ White-Box โดยมุ่งประเด็นไปที่ความถูกต้องของโครงสร้างของการวนซ้ำหรือลูป (loop constructs)

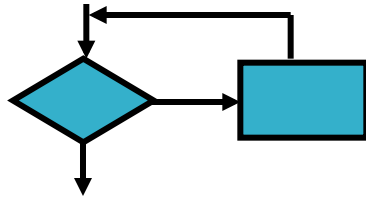
มีกลุ่มของลูปที่แตกต่างกัน มีอยู่ 4 แบบ ได้แก่

1. ลูปอย่างง่าย
2. ลูปซ้อนลูป
3. ลูปต่อกัน
4. ลูปไม่เป็นโครงสร้าง

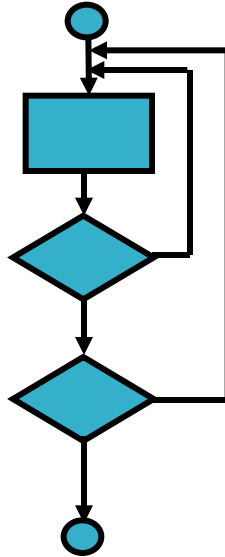
Loop Testing

1. Simple Loop เป็นรูปแบบง่ายไม่มีความซับซ้อน
2. Nested loop เป็นลูปที่มีโครงสร้างลักษณะนำมาซ้อนกันภายในมีความซับซ้อนมากขึ้น
3. Concatenated loop เป็นลูปที่มีการนำมาต่อกันเป็นโครงสร้างใหญ่ขึ้น
4. Unstructured loops เป็นลูปที่ไม่มีโครงสร้างของลูปชัดเจนมีความซับซ้อนการทดสอบมีความยุ่งยากมาก

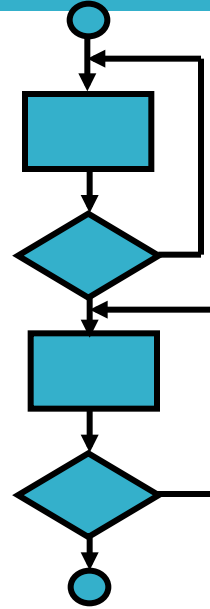
Loop Testing



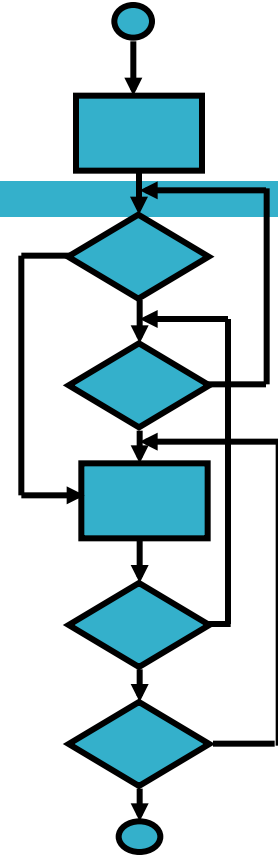
Simple loops



Nested loops



Concatenated loops



Unstructured loops

Loop Testing

- ชุดทดสอบลูปอย่างง่าย เมื่อ n เป็นจำนวนรอบสูงสุดที่อนุญาตให้ผ่านลูป
 1. ทำงานข้ามลูปไปเลย
 2. ทำงานผ่านลูปเพียงหนึ่งรอบ
 3. ทำงานผ่านลูป 2 รอบ
 4. ทำงานผ่านลูป m รอบ เมื่อ $m < n$
 5. ทำงานผ่านลูป $n-1, n, n+1$ รอบ

Loop Testing

ในการทดสอบแบบลูปซ้อนลูป จะทำให้การทดสอบเพิ่มขึ้นเป็นตามจำนวนชั้นของลูปที่ซ้อน มีข้อแนะนำในการทดสอบ

- ❑ เริ่มทดสอบจากลูปในสุดก่อน โดยตั้งค่าลูปอื่นๆ ให้มีค่าน้อยที่สุด
- ❑ ทำการทดสอบจากลูปในไปลูปนอกทีละชั้น
- ❑ ทำต่อไปจนกว่าทุกๆ ลูปได้ทดสอบหมด

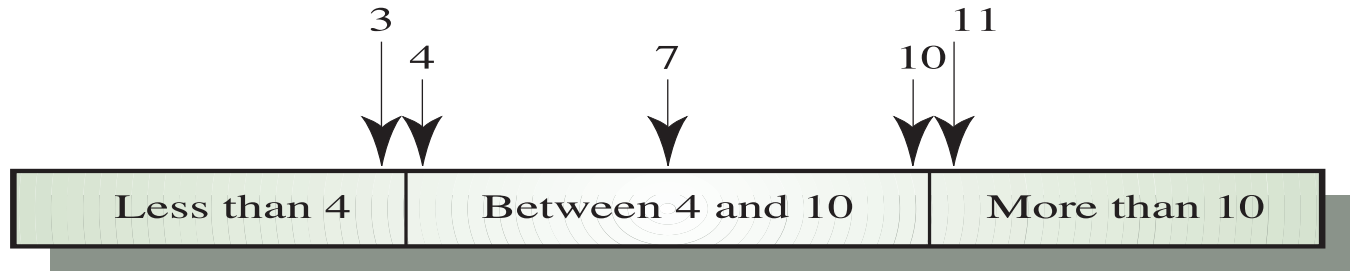
การทดสอบแบบกล่องดำ (Black box)

- เรียกอีกอย่างหนึ่งว่า “การทดสอบเชิงพฤติกรรม (behavioral testing)” โดยมีเป้าหมายของการทดสอบที่ความต้องการของโปรแกรมนั้นๆ ซึ่งจะไม่พิจารณาถึงโครงสร้างของโปรแกรม เพื่อทดสอบสิ่งต่อไปนี้
 - ▣ หน้าที่เกิดผิดพลาดหรือหายไป
 - ▣ ความผิดพลาดจากตัวต่อประสาน (หน้าขาดหายไป)
 - ▣ ความผิดพลาดในโครงสร้างข้อมูล หรือเข้าถึงฐานข้อมูลภายนอก
 - ▣ ความผิดพลาดเชิงพฤติกรรมหรือเกณฑ์ประกอบการ (ประมวลผลข้อมูล)
 - ▣ ความผิดพลาดจากการเริ่มต้นหรือจบงาน

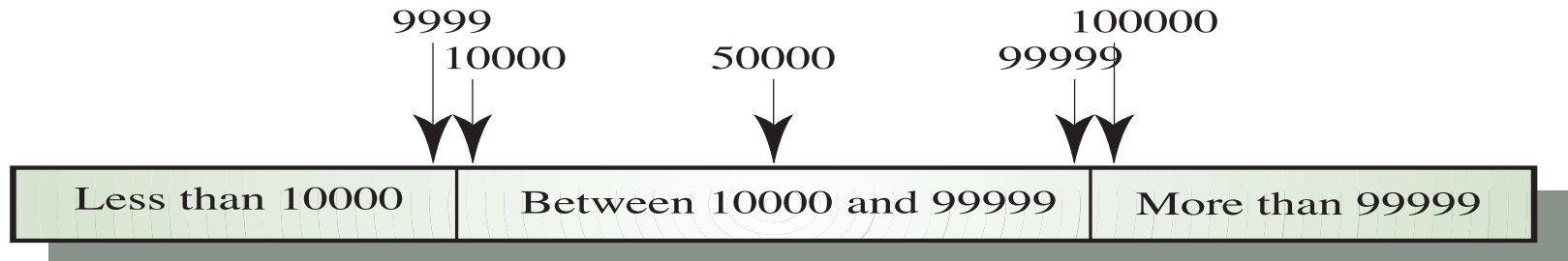
การทดสอบแบบกล่องดำ (Black box)

- การแบ่งส่วนสมมูล (Equivalence Partitioning) เป็นวิธีการทดสอบแบบกล่องดำ ที่มีการแบ่งส่วนข้อมูลนำเข้าออกเป็นกลุ่มๆ โดยมีข้อแนะนำในการแบ่งข้อมูลดังนี้
 1. ค่าข้อมูลต่ำสุดของพิสัยข้อมูล (ขอบเขตข้อมูล)
 2. ค่าข้อมูลสูงสุดของพิสัยข้อมูล
 3. ค่าข้อมูลตัวแทนกลุ่ม เป็นค่าที่ใกล้เคียงกับค่ากลางของพิสัยข้อมูล
 4. ค่าข้อมูลเกินพิสัย หรือเกินขอบเขตข้อมูลในแต่ละช่วง

การแบ่งส่วนสมมูล (Equivalence Partitioning)



Number of input values



Input values